Synchronization Hardware

What is it?

Synchronization hardware provides special CPU instructions that help processes and threads access shared resources safely.

- It supports process synchronization at the hardware level.
- It prevents multiple processes from modifying shared data simultaneously.
- It helps avoid race conditions.
- It improves system efficiency and performance.

Semaphores

What is it?

A Semaphore is a synchronization tool used to control access to shared resources by multiple processes or threads.

- It uses a counter to manage resource access.
- It helps coordinate multiple processes.
- It prevents race conditions.
- It uses two operations: Wait() and Signal().

Mutex Locks

What is it?

A Mutex Lock is a locking mechanism that allows only one process or thread to use a shared resource at a time.

- Mutex stands for Mutual Exclusion.
- Only one thread can hold a mutex lock at a time.
- A thread must lock before entering the critical section.
- The lock is released after the task is completed.
- It prevents data inconsistency and conflicts.

Classic Problems of Synchronization

What is it?

Classic synchronization problems are standard examples used to understand synchronization techniques in operating systems.

- They demonstrate common synchronization challenges.
- Producer-Consumer Problem involves sharing a buffer between producer and consumer.
- Readers-Writers Problem manages access to shared data.
- Dining Philosophers Problem deals with resource sharing among multiple processes.
- These problems help explain synchronization concepts and solutions.

Interprocess Communication

- **Interprocess Communication (IPC)** is a mechanism that allows processes to **communicate and share data** with each other in an operating system.
- In a **multi-process** system, processes often need to share **information, synchronize their actions, or communicate** with each other to perform tasks.
- IPC provides the means for these **interactions**.

Why Synchronization is important

1. **Process Independence**: Each process **runs independently**, but many tasks require them to **cooperate and share data**.
2. **Concurrency**: IPC is crucial for processes **running concurrently on a CPU**, as it allows them to communicate without interfering with one another.
3. **Synchronization**: IPC helps synchronize processes, ensuring that they can access shared resources without **causing conflicts or inconsistencies.**

Types of Interprocess Communication

- There are several mechanisms for inter-process communication, each suitable for different scenarios. Below are the common types:

1. Message Passing

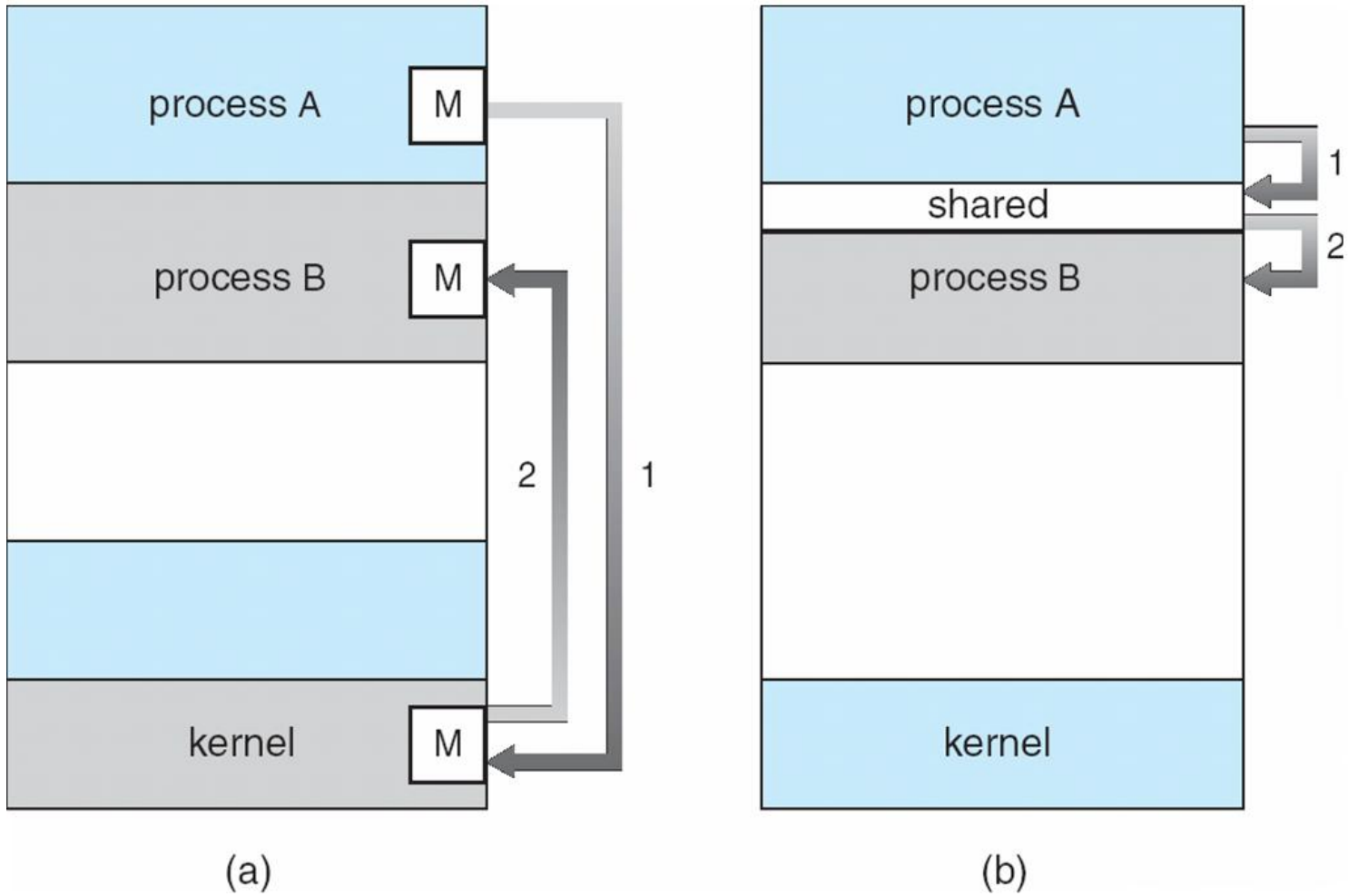
- Message passing is a form of IPC where processes communicate by **sending messages** to each other. The messages can contain data or control information.
- **How it works**:
- **Direct Communication:** The sender knows the recipient's identity and sends the message directly.
- **Indirect Communication:** A message queue or mailbox is used, and processes send messages to it without knowing the recipient.

2. Shared Memory

- Shared memory is an IPC method where multiple processes can **access the same region of memory**.
- Processes can read from and write to this shared memory location to exchange information.

- Think of **sharing a whiteboard** between colleagues. Each person can write their thoughts on the board (shared memory), and everyone can see and update the information.

Communications Models



Inter Process Communication

1. **Independent process**

- A process that does not interact with other processes. It executes in isolation, meaning its execution is not affected by or does not affect the execution of any other process. e.g: text editor like notepad, web browsers and media players.

2. **Co-operating process**

- A process that can communicate and synchronize with other processes.
- These processes can share data and resources, allowing them to work together to accomplish a task or achieve a common goal. e.g: Google Docs.

LPC (Local Procedure Call)

- **Definition:** A mechanism that allows a process to raise a procedure or function in another process within the same system. It is a form of inter-process communication (IPC).
- **Purpose:** LPC enables communication between different processes, facilitating the sharing of data and services without the overhead of more complex IPC methods.
- **Characteristics**:
- **Example**: Commonly used in systems like Windows, where it allows applications to communicate with system services or other applications.
- LPC is crucial for efficient process communication and helps maintain system performance and responsiveness.

General examples of LPC

1. **Car Maintenance**

- When performing maintenance on a car:
- Local Procedure: Changing the oil.
- Example: The steps include draining old oil, replacing the oil filter, and adding new oil. Each step is a local procedure in the larger process of maintaining the vehicle.

2. **Exercise Routine**

- In a fitness program:
- Local Procedure: Doing push-ups.
- Example: Part of a workout might include a section that says, "Complete three sets of 10 push-ups." This is a local procedure within the entire exercise routine.

RPC (Remote Procedure Call)

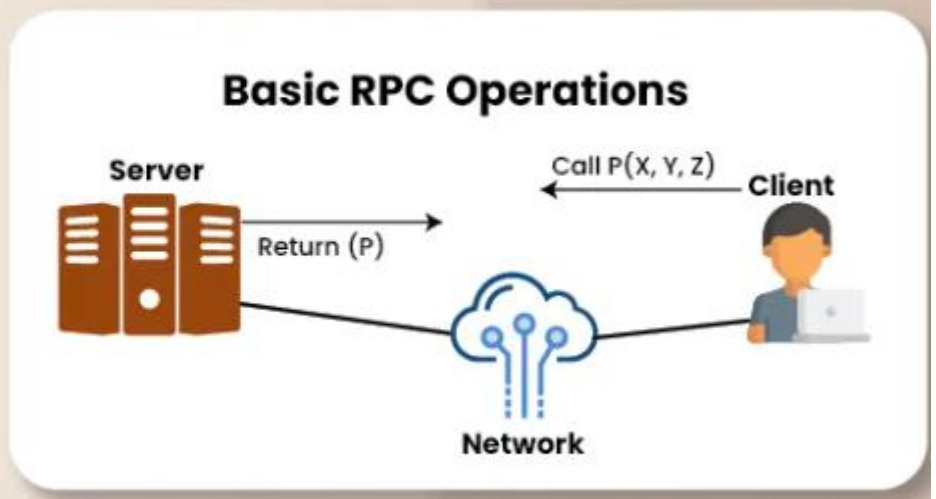
- RPC is a protocol that allows a program to call a function on a different computer (server) as if it were calling a function in its own code. The program sends a request over the network to the server, which then executes the function and sends the result back.

- Think of RPC like making a phone call to a friend to ask them to do something for you. Instead of doing the task yourself, you ask your friend (who is not in the same place as you) to do it. You tell them what you need, they do it, and then they let you know the result.

- In the same way, with RPC, your program can "ask" a different program running on another computer to do something, and it will get the results back, making it seem like everything is happening locally, even though it's happening remotely.

Remote Procedural Call (RPC) Mechanism

in Distributed Systems



How RPC takes place?

1. **Client-Server Model**

- Client: The program that makes the request for a service.
- Server: The program that provides the service or executes the requested function.

2. **Communication Protocols**

- RPC uses specific communication protocols to enable the client and server to communicate over a network. Common protocols include:

- HTTP: Often used for web services.
- TCP/IP: A foundational protocol for network communications.

SOCKETs

- A **socket** is a communication **endpoint** used to send and receive data between processes, either within the same system or over a network.
- It allows applications to exchange information seamlessly.
- it is assigned an **IP address and a port number** to establish communication.
- The server listens for incoming connections, while the client initiates communication.

Servers Services

1. Telnet: A telnet Server listens to Port number 23. (Remote Login)
2. FTP: Server listen to port number 21.
3. HTTP – Server listens to port number 80.
4. All ports below 1024 are considered as well known.